

1.1 — Análisis del problema "Div-Soup"

¿Qué etiquetas HTML5 semánticas están completamente ausentes?

<header>

<nav>

<main>

<footer>

<section>

<article>

<aside>

¿Por qué el uso excesivo de <div> es un problema para los motores de búsqueda (SEO)? ¿Y para los lectores de pantalla (accesibilidad)?

Porque los motores de búsqueda utilizan la estructura semántica para entender mejor el contenido utilizando etiquetas, pero en este caso al haber tantos div se pierde el significado semántico y además al utilizar únicamente <div>, los lectores de pantalla no pueden reconocer estas secciones automáticamente a como lo harían utilizando etiquetas.

¿Qué es el Div-Soup? Este es un término utilizado para describir cuando una pagina esta creada únicamente con div sin usar etiquetas, como resultado de esto se crea un código difícil de mantener con menor accesibilidad, peor Seo y menor legibilidad para otros desarrolladores.

1.2 — Análisis del Formulario

¿Se usan etiquetas <label> correctamente vinculadas a sus campos mediante los atributos for e id? No, solo aparecen textos descriptivos utilizando elementos <div> como el la siguiente imagen:

```
<div>Nombre completo:</div>
<input type="text" placeholder="Ingrese su nombre completo">

<div>Número de cédula:</div>
<input type="text" placeholder="X-XXXX-XXXX">

<div>Correo electrónico institucional:</div>
<input type="text" placeholder="usuario@utn.ac.cr">

<div>Número de teléfono:</div>
<input type="text" placeholder="XXXX-XXXX">
```

Al no tener los label se genera: Menor accesibilidad, los lectores de pantalla no identifican correctamente el propósito de cada campo y el usuario debe hacer clic exactamente sobre el campo y no sobre el texto asociado.

¿Qué elemento semántico debería agrupar los datos personales y cuál debería agrupar las preferencias del congreso? ¿Qué etiqueta describe el propósito de cada grupo?

Los datos personales deberían agruparse mediante `<fieldset>` con un `<legend>` y las preferencias del congreso deberían estar en otro grupo independiente. Ejemplo:

```
<fieldset>  
<legend>Datos Personales</legend>
```

```
<fieldset>  
<legend>Preferencias del Congreso</legend>
```

El campo "Carrera o Departamento" utiliza un `<select>`. ¿Por qué un `<datalist>` sería técnicamente superior para este caso?

Sería mejor utilizar el `<datalist>` porque este permite escribir texto, buscar rápidamente una carrera y seleccionar una opción existente, mejorando la experiencia del usuario

1.3 Análisis de la tabla de horarios

¿Cuáles secciones estructurales de la tabla están ausentes? ¿Por qué `<thead>`, `<tbody>` y `<tfoot>` no son meramente estéticos?

Falta el `<thead>`, `<tbody>` y el `<tfoot>`

```
<table>  
  <tr>  
    <th>Hora</th>  
    <th>Lunes 15 Ago</th>  
    <th>Martes 16 Ago</th>  
    <th>Miércoles 17 Ago</th>  
  </tr>  
  <tr>  
    <td>8:00 - 9:00</td>  
    <td>Acreditación y Bienvenida</td>  
    <td>Apertura del día</td>  
    <td>Apertura del día</td>  
  </tr>  
  <tr>  
    <td>9:00 - 10:30</td>  
    <td>Conferencia Magistral: Dra. Rodríguez</td>  
    <td>Taller de Machine Learning</td>  
    <td>Panel: Futuro Verde</td>  
  </tr>  
</table>
```

estos son importantes ya que permiten dividir las tablas en encabezado, cuerpo y pie, haciendo más fácil la interpretación para los lectores de pantalla, mejor mantenimiento y mayor accesibilidad

Los encabezados `<th>` no tienen atributo `scope`. ¿Qué valor debería tener cada uno (`col`, `row`, `colgroup`)? ¿Qué tecnología asistiva se ve afectada directamente?

Los encabezados superiores de la tabla deberían utilizar: `scope="col"` porque estas representan columnas y si existieran encabezados laterales, deberían utilizar: `scope="row"`. La principal tecnología afectada son los lectores de pantalla, como NVDA, JAWS y VoiceOver ya in el atributo `scope`, estos programas tienen más dificultades para asociar correctamente cada dato con su encabezado correspondiente.

¿Hay un elemento `<caption>` presente? ¿Para qué sirve y por qué es importante desde el punto de vista de accesibilidad (WCAG 2.1)?

No. La tabla no contiene ningún elemento `<caption>`.

El elemento `<caption>` sirve para describir el propósito general de la tabla antes de que el usuario lea su contenido.

Su uso es importante para cumplir con las recomendaciones de accesibilidad WCAG 2.1. (estándar internacional de accesibilidad digital para personas con discapacidades)

1.4 — Análisis del Elemento Video

¿Qué atributo fundamental falta para que el video muestre una imagen previa antes de que el usuario lo reproduzca? ¿Cómo impacta esto en la Cumulative Layout Shift (CLS) de las métricas Core Web Vitals?

El video no posee el atributo `poster`, este atributo permite mostrar una imagen previa antes de que el usuario reproduzca el contenido.

```
<div class="card">
  <h2>Video Promocional</h2>
  <div>
    <div class="mediaplugin mediaplugin_videojs d-block"><div style="m
    </video></div></div>
    <div>Si no puedes ver el video, actualiza tu navegador.</div>
  </div>
</div>
```

La ausencia de una imagen de vista previa puede provocar movimientos inesperados durante la carga del contenido multimedia afectando el CLS como el core web vitals ya que estas son métricas para evaluar la experiencia de usuario

¿Por qué es problemático tener un solo `<source>`? ¿Qué formatos alternativos debería incluir para garantizar compatibilidad cruzada entre navegadores?

Actualmente el video dispone únicamente de un archivo MP4, si un navegador presenta problemas de compatibilidad con ese formato, el usuario no podrá reproducir el contenido, por esta razón es recomendable incluir múltiples formatos para garantizar compatibilidad entre diferentes navegadores y dispositivos. Como formato adicional para garantizar la compatibilidad está el WebM (video/webm)

**¿Qué atributo debe agregarse para que el usuario pueda controlar la reproducción?
¿Cuál es la implicación de accesibilidad de su ausencia?**

El atributo que debe agregarse es el controls ya que su ausencia impide que el usuario pueda reproducir, pausar o controlar el volumen del video de manera sencilla.

1.5 Análisis del iFrame del Mapa

¿Qué atributo de rendimiento falta y que haría que el mapa no cargue hasta que el usuario se desplace hacia él? ¿Cuál es su valor correcto?

loading="lazy" este atributo permite que el mapa se cargue únicamente cuando el usuario se aproxima a esa sección de la página.

¿Qué atributo de seguridad falta que permite al contenido embebido ejecutar scripts con los mismos permisos que la página padre? Nombre el atributo y al menos tres restricciones que debería activar.

Sandbox este atributo limita las acciones que puede realizar el contenido embebido dentro del iframe, algunas restricciones son: allow-scripts, allow-same-origin, allow-popups

¿Falta el atributo `title` en el iframe? ¿Para qué lo necesitan los lectores de pantalla?

Sí, el iframe no posee un atributo title, este atributo es utilizado por los lectores de pantalla para describir el contenido del iframe y permitir que los usuarios comprendan su propósito.

1.6 Análisis del Anidamiento Incorrecto

Identifique todas las instancias donde un elemento de bloque está anidado dentro de un elemento de línea. Cite las líneas exactas del código.

Se identificaron 2 casos, uno:

```
<span>  
<div>  
<h3>Contacto Principal</h3>  
</div>  
</span>
```

```
Dos: <a href="https://www.utn.ac.cr">  
<div>  
<div>Visitar sitio oficial de la UTN</div>  
</div>  
</a>
```

Explique por qué el navegador intenta "corregir" este error automáticamente y cómo ese proceso puede romper el layout de maneras impredecibles.

Los navegadores modernos intentan reparar automáticamente el código HTML inválido para poder mostrar la página al usuario, sin embargo, durante este proceso pueden modificar la estructura original del DOM, generando comportamientos inesperados o diferencias entre navegadores.

¿Cómo puede verificar este tipo de error usando las DevTools del navegador? Describa el procedimiento.

Para verificar estos problemas se debe abrir la página en Google Chrome o Microsoft Edge y presionar la tecla F12 para acceder a las herramientas de desarrollador.

Posteriormente, en la pestaña "Elements", se debe localizar el elemento problemático y observar cómo el navegador reorganizó automáticamente el árbol DOM.

Esto permite identificar si la estructura final coincide o no con el código fuente original.